

Claude Certified Architect Foundations

Exam Preparation Tutorial

A 101-level guide to the scenarios, domains, task statements, and concepts — written for non-developers (TPMs, program managers, and AI-curious builders)

Contents

1. How to Use This Tutorial	4
2. The Exam at a Glance	5
2.1 Format and Scoring	5
2.2 The Five Domains and Their Weight.....	5
3. The Six Exam Scenarios, Explained	7
Scenario 1: Customer Support Resolution Agent.....	7
Scenario 2: Code Generation with Claude Code	7
Scenario 3: Multi-Agent Research System	7
Scenario 4: Developer Productivity with Claude	7
Scenario 5: Claude Code for Continuous Integration (CI/CD)	8
Scenario 6: Structured Data Extraction.....	8
3.1 Scenario-to-Domain Map.....	8
4. Core Concepts Primer (Read This Before the Domains)	9
4.1 The Absolute Basics (Start Here if You're New).....	9
What is a large language model (LLM)?.....	9
Prompt vs. system prompt	9
What is an API?	9
Deterministic vs. probabilistic — the exam's favorite distinction	9
Version control, repositories, and "cloning"	9
JSON in 60 seconds	10
Glob patterns — wildcards for file names	10
YAML frontmatter — the settings block at the top of a file	10
4.2 The Exam Glossary	10
5. Domain 1: Agentic Architecture & Orchestration (27%)	13
Task 1.1 — Design and Implement Agentic Loops	13
Task 1.2 — Orchestrate Multi-Agent Systems (Coordinator–Subagent)	14
Task 1.3 — Configure Subagent Invocation, Context Passing, and Spawning.....	15
Task 1.4 — Multi-Step Workflows: Enforcement and Handoff Patterns.....	16
Task 1.5 — Agent SDK Hooks: Interception and Normalization.....	17
Task 1.6 — Task Decomposition Strategies	17
Task 1.7 — Session State, Resumption, and Forking.....	18
6. Domain 2: Tool Design & MCP Integration (18%).....	19
Task 2.1 — Design Effective Tool Interfaces	19
Task 2.2 — Structured Error Responses for MCP Tools.....	20

Task 2.3 — Distribute Tools Across Agents and Configure tool_choice.....	20
Task 2.4 — Integrate MCP Servers into Claude Code and Agent Workflows.....	21
Task 2.5 — Built-In Tools: Read, Write, Edit, Bash, Grep, Glob.....	21
7. Domain 3: Claude Code Configuration & Workflows (20%)	23
Task 3.1 — CLAUDE.md Hierarchy, Scoping, and Modular Organization	23
Task 3.2 — Custom Slash Commands and Skills	24
Task 3.3 — Path-Specific Rules (Conditional Convention Loading).....	24
Task 3.4 — Plan Mode vs. Direct Execution	25
Task 3.5 — Iterative Refinement Techniques	25
Task 3.6 — Claude Code in CI/CD Pipelines	26
8. Domain 4: Prompt Engineering & Structured Output (20%).....	27
Task 4.1 — Explicit Criteria to Reduce False Positives.....	27
Task 4.2 — Few-Shot Prompting.....	27
Task 4.3 — Structured Output via Tool Use and JSON Schemas.....	28
Task 4.4 — Validation, Retry, and Feedback Loops	28
Task 4.5 — Batch Processing Strategies.....	29
Task 4.6 — Multi-Instance and Multi-Pass Review Architectures.....	29
9. Domain 5: Context Management & Reliability (15%)	31
Task 5.1 — Preserve Critical Information Across Long Interactions.....	31
Task 5.2 — Escalation and Ambiguity Resolution.....	31
Task 5.3 — Error Propagation Across Multi-Agent Systems	32
Task 5.4 — Context Management in Large Codebase Exploration	32
Task 5.5 — Human Review Workflows and Confidence Calibration	33
Task 5.6 — Information Provenance and Uncertainty in Multi-Source Synthesis	33
10. Sample Question Walkthroughs: Learning the Exam's Logic	35
11. Putting It to Work: Claude Code Habits for Non-Developers	37
12. One-Page Cheat Sheet: Facts to Memorize	38
13. A Two-Week Study Plan	40

1. How to Use This Tutorial

This tutorial translates the official **Claude Certified Architect – Foundations** exam guide into plain language. You do not need to write code to follow it. If you can read a process diagram, understand what an API roughly does, and have managed technical teams or products, you have everything you need. And if even those terms feel shaky, **Section 4.1 ("The Absolute Basics")** builds everything up from zero — start there.

The exam is not a trivia test. Almost every question is a **judgment call**: "Given this production problem, which of these four fixes is the right one?" The wrong answers are usually things that *sound* reasonable — adding more instructions, buying a bigger model, building extra infrastructure — while the right answer fixes the actual root cause with the simplest reliable mechanism. Internalizing that mindset matters more than memorizing any single fact.

Three recurring themes show up across all five domains. If you remember nothing else, remember these:

- **1. Guarantees beat instructions.** When a rule **MUST** be followed (e.g., verify identity before issuing a refund), enforce it in code (hooks, gates, schemas), not in the prompt. Prompts are probabilistic — they fail a small percentage of the time, and that is unacceptable for money, security, or compliance.
- **2. Fix the root cause, proportionately.** Vague tool descriptions? Improve the descriptions — don't build a routing classifier. Unclear escalation rules? Write explicit criteria with examples — don't deploy a second ML model. The exam loves "over-engineered" distractors.
- **3. Context is a scarce, leaky resource.** Claude's working memory (the "context window") fills up, important details get buried in the middle, and summaries lose precise numbers. Good architects extract key facts, trim noise, and pass information explicitly between agents.

How to study: Read this tutorial once end-to-end, then re-read the domain chapters alongside hands-on practice in Claude Code. Finish with the sample-question walkthroughs (Chapter 10) and the practice exam.

2. The Exam at a Glance

The certification validates that you can make **informed tradeoff decisions** when building real solutions with Claude across four core technologies: **Claude Code** (the agentic coding tool), the **Claude Agent SDK** (for building your own agents), the **Claude API** (direct programmatic access), and **MCP** — the Model Context Protocol, an open standard for connecting Claude to external systems.

2.1 Format and Scoring

Item	Details
Question format	Multiple choice — one correct answer, three distractors designed to tempt candidates with incomplete knowledge
Scenarios	4 scenarios per exam, drawn at random from a pool of 6 published scenarios; questions are framed within those scenarios
Scoring	Scaled score from 100 to 1,000; minimum passing score is 720
Result	Pass / Fail, judged against a standard set by subject matter experts
Guessing	No penalty — unanswered questions count as wrong, so always answer every question
Target candidate	Solution architect with ~6+ months of hands-on experience building with Claude

2.2 The Five Domains and Their Weight

Five content domains are tested. Weighting tells you where to invest study time — Domain 1 alone is over a quarter of the exam.

Domain	Weight	One-line summary
1. Agentic Architecture & Orchestration	27%	How agents loop, decide, delegate to sub-agents, and enforce workflows
2. Tool Design & MCP Integration	18%	How to design, describe, distribute, and error-handle the tools agents use
3. Claude Code Configuration & Workflows	20%	CLAUDE.md files, rules, slash commands, skills, plan mode, CI/CD integration
4. Prompt Engineering & Structured Output	20%	Precise criteria, few-shot examples, JSON schemas, validation/retry, batching
5. Context Management & Reliability	15%	Preserving key facts, escalation logic, error propagation, human review

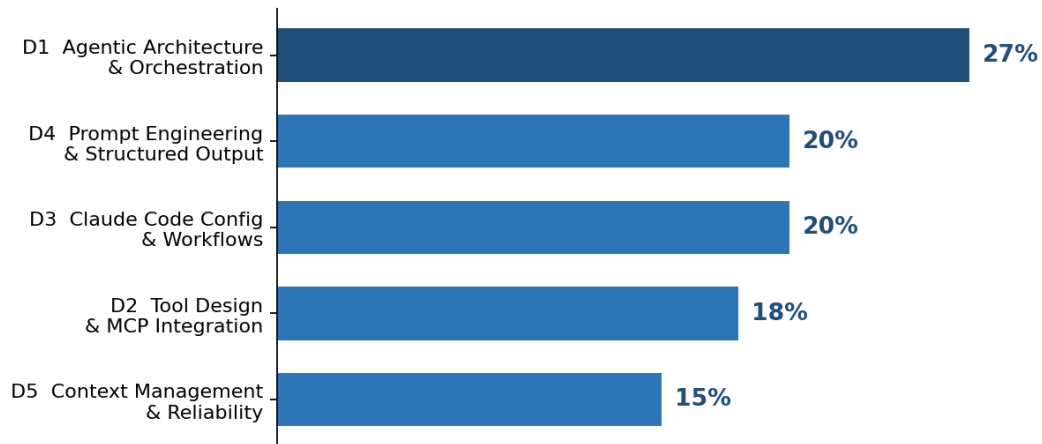


Figure 1 — Where the exam points sit: Domain 1 alone is over a quarter of all questions.

Exam tip: Domains 1 + 2 together are 45% of the exam and share one philosophy: agents are only as reliable as the structure around them — tool descriptions, programmatic gates, and explicit context passing. Master those two domains first.

3. The Six Exam Scenarios, Explained

Every exam question lives inside one of six realistic production scenarios. You will see **4 of the 6** on your exam, chosen at random. Understanding each scenario's "shape" — what is being built, what can go wrong, and which domains it tests — lets you anticipate the kinds of questions you'll face.

Scenario 1: Customer Support Resolution Agent

What it is: An AI agent (built with the Claude Agent SDK) that resolves customer requests — returns, billing disputes, account issues — end to end. It talks to backend systems through four custom MCP tools: `get_customer`, `lookup_order`, `process_refund`, and `escalate_to_human`. The business target is 80%+ first-contact resolution *while knowing when to hand off to a person*.

Analogy: Think of a new support rep with a great manual but no supervision. The architecture questions are about which guardrails you bolt on: identity checks before refunds, refund caps, and clear rules for when to call a supervisor over.

Typical failure modes tested: skipping identity verification, refunding the wrong account, escalating easy cases while bungling hard ones, losing track of order numbers in long chats.

Scenario 2: Code Generation with Claude Code

What it is: A development team adopting Claude Code for everyday work — generating, refactoring, debugging, and documenting code. Questions focus on team configuration: custom slash commands, CLAUDE.md files, and when to use **plan mode** (explore and design first) versus **direct execution** (just make the change).

Analogy: This is the "team enablement" scenario — like rolling out a powerful new tool across an engineering org and deciding which settings live in the shared playbook (version control) versus each person's personal preferences.

Scenario 3: Multi-Agent Research System

What it is: A **coordinator** agent that delegates to specialist **subagents** — one searches the web, one analyzes documents, one synthesizes findings, one writes the report. The output is a comprehensive, properly cited research report.

Analogy: A research director managing a team of analysts. The director splits the topic into assignments, routes information between analysts (who do not talk to each other directly), checks the draft for gaps, and sends people back for more digging.

Typical failure modes tested: the coordinator slicing the topic too narrowly (so whole areas are never researched), losing source citations during summarization, and one analyst's timeout killing the whole project.

Scenario 4: Developer Productivity with Claude

What it is: An Agent SDK-based assistant that helps engineers explore unfamiliar codebases, understand legacy systems, generate boilerplate, and automate repetitive tasks. It uses Claude's built-in tools — `Read`, `Write`, `Bash`, `Grep`, `Glob` — plus MCP servers.

Analogy: A consultant dropped into a 10-year-old company with no org chart. The questions test search strategy: do you read every document (wasteful), or search for entry points and follow the threads (efficient)?

Scenario 5: Claude Code for Continuous Integration (CI/CD)

What it is: Claude Code wired into the automated pipeline that runs whenever developers submit code: automated code reviews, test generation, and pull-request feedback. The challenge is producing **actionable feedback with few false positives** — and running Claude non-interactively (no human at the keyboard).

Analogy: An automated quality inspector on an assembly line. If it flags too many non-issues, workers start ignoring it entirely — so precision matters more than volume.

Scenario 6: Structured Data Extraction

What it is: A system that reads unstructured documents (invoices, contracts, reports) and outputs clean, structured data validated against **JSON schemas** — a machine-readable template that defines exactly which fields exist and what types they hold. It must handle edge cases gracefully and feed downstream systems reliably.

Analogy: Converting a shoebox of paper receipts into a perfectly formatted spreadsheet — including deciding what to do when a receipt is smudged (mark it "unknown") rather than inventing a number.

3.1 Scenario-to-Domain Map

Scenario	Primary domains tested
1. Customer Support Agent	D1 Orchestration · D2 Tools/MCP · D5 Context & Reliability
2. Code Generation with Claude Code	D3 Claude Code Config · D5 Context & Reliability
3. Multi-Agent Research System	D1 Orchestration · D2 Tools/MCP · D5 Context & Reliability
4. Developer Productivity	D2 Tools/MCP · D3 Claude Code Config · D1 Orchestration
5. Claude Code for CI/CD	D3 Claude Code Config · D4 Prompting & Structured Output
6. Structured Data Extraction	D4 Prompting & Structured Output · D5 Context & Reliability

4. Core Concepts Primer (Read This Before the Domains)

This chapter has two parts. **Section 4.1** explains the absolute basics from the ground up — start here if terms like "API" or "version control" are fuzzy. **Section 4.2** is the exam glossary you will refer back to throughout the domain chapters.

4.1 The Absolute Basics (Start Here if You're New)

What is a large language model (LLM)?

Software that reads text and writes text in response. You give it a message — called a **prompt** — and it produces an answer, one small chunk at a time, based on patterns learned from enormous amounts of text. Claude is an LLM built by Anthropic. Two practical consequences follow: it has no memory between conversations unless you give it one, and its answers are *very likely* correct rather than *guaranteed* correct.

Prompt vs. system prompt

The **prompt** is whatever you type. The **system prompt** is a set of standing instructions the builder of the application writes once, which silently shapes every answer — e.g., *"You are a customer support agent for Acme. Verify identity before discussing accounts. Never promise delivery dates."* When the exam says "add criteria to the system prompt," it means: change those standing instructions.

What is an API?

An **API** (Application Programming Interface) is how one piece of software talks to another over the internet — no chat window, no human. "Calling the Claude API" simply means: your company's software sends Claude a prompt as data and receives the response as data. That is how Claude gets embedded inside products, pipelines, and agents.

Analogy: A restaurant: you (the software) hand the waiter (the API) a structured order; the kitchen (Claude) prepares it; the waiter brings back the dish in a predictable format. You never walk into the kitchen.

Deterministic vs. probabilistic — the exam's favorite distinction

Deterministic = the same input produces the same outcome, every single time. A calculator, a locked door, a line of code that says "block refunds over \$500." **Probabilistic** = usually right, occasionally not — like a skilled employee following written guidance. LLMs are probabilistic; code is deterministic. Roughly a third of the exam boils down to: **this rule is too important to leave to a probabilistic mechanism — enforce it with a deterministic one.**

Version control, repositories, and "cloning"

A **repository** ("repo") is the shared project folder for a piece of software. **Version control** (the dominant tool is Git) tracks every change to that folder and synchronizes it among teammates. **Cloning** means downloading your own complete copy of the repo. The crucial consequence — which answers several exam questions by itself:

- **Anything saved inside the repo** (project CLAUDE.md, .claude/commands/, .mcp.json) automatically reaches every teammate when they clone or update.
- **Anything saved in your personal home folder** (paths starting with ~/, like ~/.claude/) stays on your machine only — teammates never see it.

JSON in 60 seconds

JSON is a simple text format for structured data — labels paired with values, readable by both humans and machines. Here is a complete JSON object describing an invoice:

```
{
  "vendor": "Acme Corp",
  "amount": 1200.50,
  "currency": "USD",
  "paid": true,
  "po_number": null
}
```

Note **null** — the explicit way to say "this value is unknown/absent," which matters enormously in Domain 4. A **JSON schema** is the rulebook for such objects: which labels must exist, what type each value must be (number, text, true/false), and which values are allowed. If the schema says **amount** is a required number, output missing it is rejected automatically.

Glob patterns — wildcards for file names

A **glob pattern** matches file names using wildcards: ***** means "anything, within one folder," and ****** means "anything, at any folder depth." Three examples cover everything the exam needs:

Pattern	Matches
*.md	Every file ending in .md in the current folder
**/*.test.tsx	Every file ending in .test.tsx, anywhere in the entire project — no matter how deeply nested
src/api/**	Everything inside the src/api folder, at any depth

YAML frontmatter — the settings block at the top of a file

Several Claude Code features (skills, path-scoped rules) are configured by a small **frontmatter** block: a few setting lines fenced between two `---` lines at the very top of a text file. Example — a rule file that loads only when editing test files:

```
---
paths: ["**/*.test.tsx"]
---
Testing conventions:
- Every test name describes the behavior being verified
- Mock external services; never call real APIs in tests
```

The lines between the dashes are machine-read settings; everything below is the human-readable instruction content.

4.2 The Exam Glossary

The exam guide assumes fluency with a set of technical terms. Here is the plain-language dictionary. Skim it now and refer back as needed.

Term	Plain-language meaning
Agent / agentic	An AI that doesn't just answer once — it works in a loop: think, take an action (call a tool), look at the result, decide the next action, repeat until done.
Tool	A capability the agent can invoke — look up an order, search the web, edit a file. Each tool has a name, a description, and defined inputs/outputs.
MCP	Model Context Protocol — an open standard ("USB-C for AI") that lets Claude connect to external systems. An MCP server exposes tools (actions) and resources (readable content catalogs).
Agentic loop / stop_reason	The control loop of an agent. After each model response, code checks stop_reason: "tool_use" means Claude wants to run a tool (keep looping); "end_turn" means Claude is finished (stop).
Coordinator / subagent	A lead agent that breaks a task into pieces and delegates each piece to a specialist agent. Subagents have isolated memory — they know only what is explicitly written into their prompt.
Hook	Code that intercepts the agent's actions — before a tool runs (to block or modify it) or after it returns (to clean up the data). Hooks give deterministic, guaranteed enforcement.
Context window	The model's working memory for a conversation — everything it can currently "see." It is finite; verbose tool output and long chats fill it up and degrade quality.
Token	The unit text is measured in (~3/4 of a word). Context limits, costs, and budgets are all counted in tokens.
CLAUDE.md	A plain-text instruction file Claude Code reads automatically — the project's standing "operating manual" (coding standards, conventions, context).
Slash command	A reusable saved prompt invoked by typing /name in Claude Code (e.g., /review runs the team's review checklist).
Skill	A folder with a SKILL.md playbook that Claude loads on demand for a specific task type; configurable via frontmatter settings (allowed tools, isolated context, argument hints).
Plan mode	A Claude Code mode where Claude explores and proposes an approach before changing anything — measure twice, cut once.
JSON / JSON schema	JSON is a structured data format; a JSON schema is the template that defines required fields, types, and allowed values — the contract for structured output.
Few-shot examples	2–4 worked examples placed in the prompt showing input → correct output, so the model learns the pattern and generalizes to new cases.
Message Batches API	A submit-now, collect-later API: 50% cheaper, results within up to 24 hours, no latency guarantee. Great for overnight jobs, wrong for anything a human is waiting on.
False positive	Flagging something as a problem when it isn't. High false-positive rates destroy user trust in automated review systems.

Term	Plain-language meaning
CI/CD	Continuous Integration / Continuous Deployment — the automated pipeline that builds, tests, and reviews code whenever developers submit changes.

5. Domain 1: Agentic Architecture & Orchestration (27%)

The biggest domain. It covers how a single agent runs its work loop, how a coordinator manages a team of subagents, and how you enforce workflows that *must* happen in a certain order.

Task 1.1 — Design and Implement Agentic Loops

What it means: An agent is a conversation between your code and Claude that repeats until the job is done. Your code sends a request; Claude replies and includes a **stop_reason** field that says why it stopped talking. There are two values you must know cold:

stop_reason	Meaning	What your code should do
"tool_use"	Claude wants a tool executed (e.g., "run lookup_order for #12345")	Execute the tool, append the result to the conversation history, and send it back — the loop continues
"end_turn"	Claude has finished and produced its final answer	Exit the loop and present the response

Why tool results go back into the conversation: Claude has no memory outside the conversation. If the tool result isn't appended to the history, Claude literally never learns what happened and cannot reason about its next step.

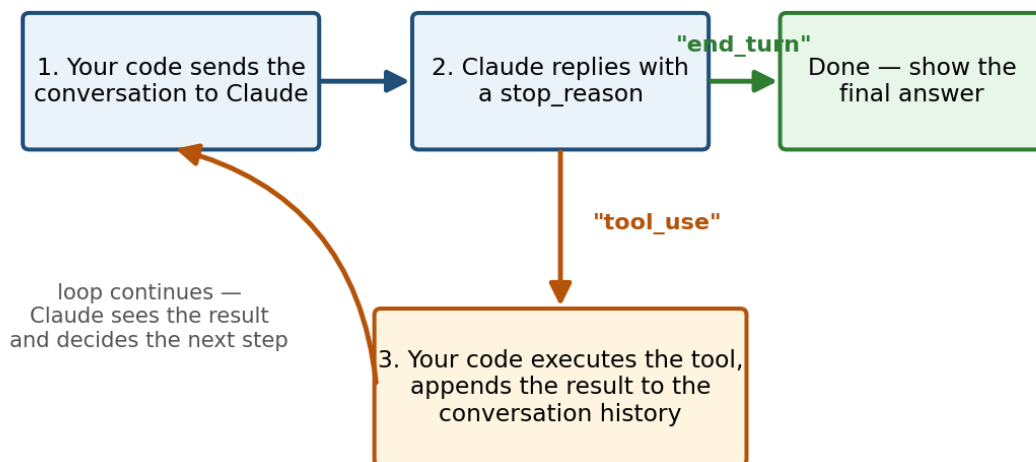


Figure 2 — The agentic loop. The orange path repeats as many times as needed; only "end_turn" exits.

Watch one complete loop in action (a support agent answering "Where is my order #12345?"):

Step	Who	What happens
1	Your code	Sends Claude the customer's message: "Where is my order #12345?"
2	Claude	Replies with stop_reason = "tool_use": "run lookup_order with order_id 12345"

Step	Who	What happens
3	Your code	Actually executes the lookup against the order system, gets back {status: "shipped", eta: "June 15"}, appends that result to the conversation, sends it to Claude
4	Claude	Now knows the facts; replies with stop_reason = "end_turn": "Good news — your order shipped and arrives June 15."
5	Your code	Sees "end_turn", exits the loop, shows the customer the answer

Steps 2–3 can repeat many times (look up the customer, then the order, then the refund policy...) before the final "end_turn." That repetition *is* the agentic loop. Note who does what: **Claude decides** which tool to use; **your code executes** it — Claude never touches your systems directly.

Model-driven vs. pre-scripted decisions: The whole point of an agent is that *Claude decides* which tool to call next based on what it has learned so far — not a fixed flowchart your team wrote in advance. Pre-configured decision trees are brittle; model-driven loops adapt.

Anti-patterns the exam will tempt you with:

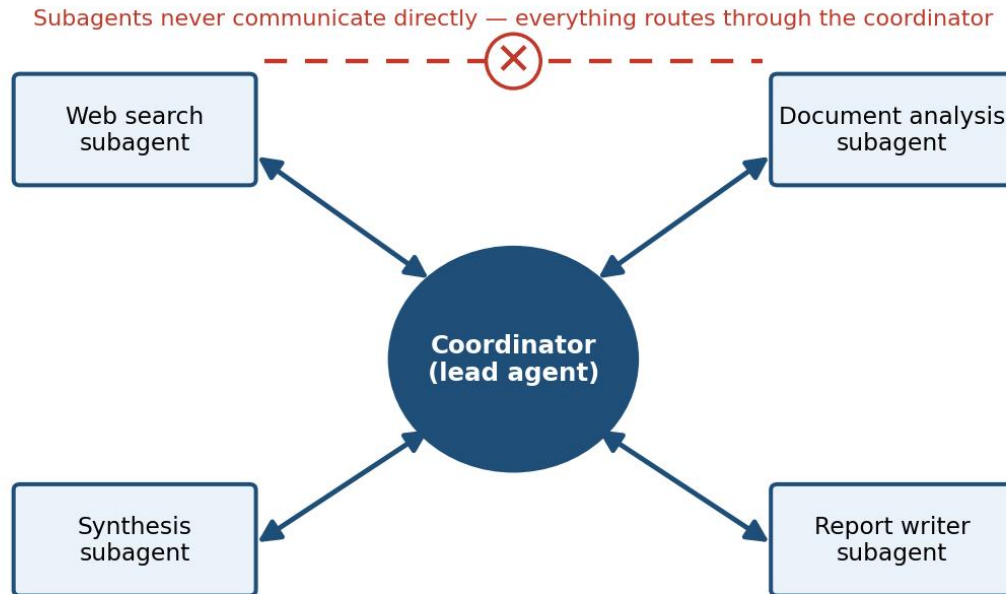
- Parsing Claude’s natural-language text for phrases like "I’m done" to decide when to stop — fragile and wrong. Use stop_reason.
- Using an arbitrary cap ("stop after 10 iterations") as the primary stopping mechanism — a safety net is fine, but it must not be how the loop normally ends.
- Checking whether the response contains text as a "completion" signal — Claude can include commentary alongside a tool call.

Exam tip: If an answer option decides loop termination by anything other than **stop_reason**, it is almost certainly a distractor.

Task 1.2 — Orchestrate Multi-Agent Systems (Coordinator–Subagent)

What it means: For big tasks, one agent coordinates several specialists. The standard architecture is **hub-and-spoke**: all communication flows through the coordinator. Subagents never talk to each other directly.

Analogy: A general contractor and trade crews. The plumber and electrician don’t coordinate with each other — the contractor sequences the work, passes information between them, handles problems, and inspects the result. That central routing gives you observability, consistent error handling, and controlled information flow.



Each subagent has isolated memory: it knows only what the coordinator writes into its prompt

Figure 3 — Hub-and-spoke: every arrow touches the coordinator; the red line marks the path that must NOT exist.

The coordinator's four jobs: (1) decompose the task into subtasks, (2) delegate to the right subagents — dynamically choosing which to invoke based on the query, not always running the full pipeline, (3) aggregate results, and (4) evaluate the output for gaps and re-delegate if coverage is insufficient (an iterative refinement loop).

Critical facts to remember:

- **Subagents have isolated context.** They do not inherit the coordinator's conversation history. Anything they need must be written explicitly into their prompt.
- **Narrow decomposition is a top failure mode.** Exam example: asked to research "impact of AI on creative industries," the coordinator created subtasks only for digital art, graphic design, and photography — so music, writing, and film were never covered. Each subagent did its job perfectly; the assignments were the problem.
- **Partition scope to avoid duplication.** Assign each subagent distinct subtopics or source types so two agents don't research the same thing.

Exam tip: When a multi-agent system produces an incomplete report but every subagent "completed successfully," look upstream: the root cause is usually the coordinator's task decomposition, not the subagents.

Task 1.3 — Configure Subagent Invocation, Context Passing, and Spawning

What it means: The mechanics of actually launching subagents in the Claude Agent SDK.

- **The Task tool spawns subagents** — and the coordinator's `allowedTools` list must include `"Task"` or it simply cannot delegate. A classic exam gotcha.
- **Context must be passed explicitly.** When the synthesis subagent needs the web-search findings, the coordinator must paste those findings into the synthesis agent's prompt. There is no shared memory.

- **AgentDefinition** is each subagent's job description: its purpose description, system prompt, and tool restrictions.
- **Parallel spawning**: to run subagents simultaneously, the coordinator emits multiple Task tool calls in a *single* response — not one per turn. This is how you cut latency.
- **Structured handoffs preserve attribution**: pass findings as structured data separating content from metadata (source URLs, document names, page numbers), so citations survive the journey between agents.
- **Goals over procedures**: coordinator prompts should state research goals and quality criteria ("find peer-reviewed sources covering all major creative sectors") rather than rigid step-by-step scripts — this lets subagents adapt.
- **fork_session** creates independent branches from a shared analysis baseline — e.g., explore two different refactoring strategies from the same codebase understanding without redoing the analysis.

Task 1.4 — Multi-Step Workflows: Enforcement and Handoff Patterns

The single most important idea in this domain:

Prompt instructions are probabilistic; programmatic enforcement is deterministic. Telling Claude "always verify the customer before refunding" works most of the time. "Most of the time" is a disaster for financial operations. When compliance must be guaranteed, build a **programmatic prerequisite gate**: code that blocks `process_refund` until `get_customer` has returned a verified customer ID.

Approach	Reliability	When to use
Prompt-based guidance ("you must do X first")	Probabilistic — non-zero failure rate	Preferences, style, soft ordering where occasional deviation is tolerable
Programmatic enforcement (hooks, prerequisite gates)	Deterministic — guaranteed	Identity verification, financial limits, compliance, security — anything where one failure is unacceptable

Prompt instruction (probabilistic) Programmatic gate (deterministic)

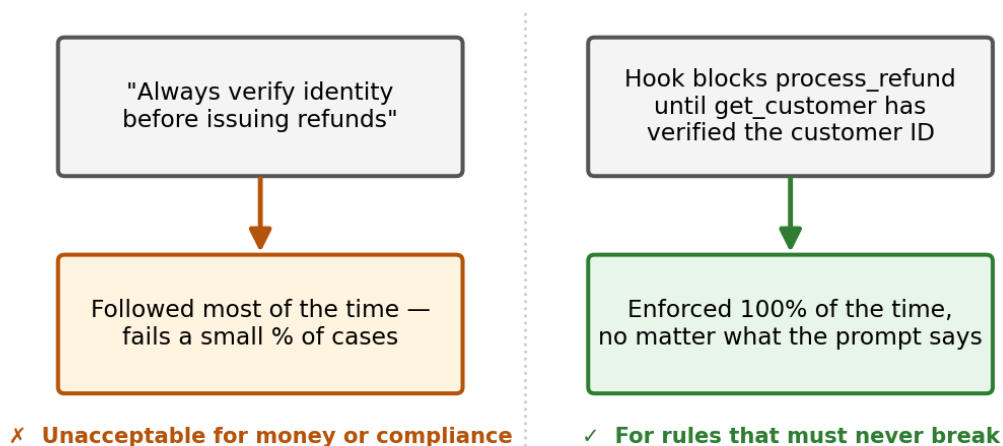


Figure 4 — The exam's most-tested distinction: guidance that usually works vs. a gate that always works.

Also in this task statement:

- **Multi-concern requests:** when a customer raises three issues in one message, decompose into distinct items, investigate each (in parallel, with shared context), then synthesize one unified resolution — don't answer only the first issue.
- **Structured handoff summaries:** when escalating to a human who cannot see the chat transcript, compile customer ID, root cause, refund amount, and recommended action. Never hand a human a cold start.

Task 1.5 — Agent SDK Hooks: Interception and Normalization

What it means: Hooks are checkpoints around tool calls. Two patterns matter:

- **PostToolUse hooks (after):** transform tool results before Claude sees them. Example: three backend systems return dates as Unix timestamps, ISO 8601 strings, and numeric status codes — a PostToolUse hook normalizes them all to one format so the agent never has to juggle inconsistencies.
- **Tool-call interception hooks (before):** inspect outgoing tool calls and block policy violations. Example: any `process_refund` call above \$500 is blocked and redirected to human escalation — guaranteed, every time, regardless of what the prompt says.

Analogy: A mailroom that opens every inbound package and standardizes the paperwork (PostToolUse), and a finance approval gate that physically cannot release payments above a threshold without a manager's signature (interception).

Exam tip: Hooks vs. prompts is the same dichotomy as Task 1.4: "guaranteed compliance required" → hook. The distractors will offer prompt wording, few-shot examples, or confidence thresholds instead.

Task 1.6 — Task Decomposition Strategies

Two decomposition styles, matched to two kinds of work:

Pattern	What it is	Best for	Example
Prompt chaining (fixed sequential pipeline)	Pre-defined sequence of focused passes	Predictable, multi-aspect work where the steps are known in advance	Code review: analyze each file individually, then run one cross-file integration pass
Dynamic adaptive decomposition	Generate the next subtasks based on what each step discovers	Open-ended investigation where you can't know the steps upfront	"Add comprehensive tests to a legacy codebase": map the structure first, find high-impact areas, build a prioritized plan that adapts as dependencies surface

Why split big reviews into passes? Attention dilution. Asking the model to review 14 files at once produces uneven depth and contradictory findings. Per-file passes give consistent local analysis; a separate integration pass catches cross-file issues.

Task 1.7 — Session State, Resumption, and Forking

- `--resume <session-name>` continues a specific named prior conversation — useful for multi-day investigations.
- `fork_session` branches a session into independent explorations from a shared baseline — e.g., compare two testing strategies without repeating the shared codebase analysis.
- **Stale context is dangerous:** if files changed since the last session, tell the resumed agent exactly which files changed so it re-analyzes only those. If prior tool results are largely stale, **start a fresh session and inject a structured summary** instead — more reliable than resuming a session full of outdated facts.

Situation	Right move
Prior context still mostly valid; a few files changed	Resume the session; explicitly list the changed files for targeted re-analysis
Prior tool results largely outdated	Start a new session seeded with a structured summary of validated findings
Want to compare two divergent approaches from one analysis	<code>fork_session</code> into parallel branches

6. Domain 2: Tool Design & MCP Integration (18%)

Agents choose tools by reading their descriptions — nothing more. This domain is about making those descriptions unambiguous, structuring errors so agents can recover intelligently, giving each agent only the tools it needs, and configuring MCP servers correctly.

Task 2.1 — Design Effective Tool Interfaces

The core principle: tool descriptions are the primary mechanism for tool selection. If two tools have minimal, near-identical descriptions ("Retrieves customer information" vs. "Retrieves order details"), the model will misroute between them — and no amount of clever prompting fixes the root cause.

A good description includes: the tool's purpose, accepted input formats, example queries, edge cases, and explicit boundaries ("use this for X; for Y, use tool Z instead").

Analogy: Two unlabeled doors marked "Service" and "Support" — people pick randomly. Label them "Returns & refunds (have your order number)" and "Account & billing questions" and misrouting disappears.

See the difference for yourself:

Quality	Tool description
Weak (causes misrouting)	"Retrieves customer information."
Strong	"Looks up a customer profile by email address or customer ID (format: CUST-12345). Returns name, account status, loyalty tier, and contact details. Example query: 'find the account for jane@example.com'. If the question is about a specific purchase or delivery, use lookup_order instead — this tool has no order data."

The strong version states purpose, input formats, an example, and — critically — an explicit boundary pointing to the neighboring tool. The model reads exactly this text when deciding which tool to call; richer text means better decisions.

Remedies tested on the exam:

- **Expand descriptions first** — the low-effort, high-leverage fix and the correct "first step" in the official sample question.
- **Rename to eliminate overlap** — e.g., a vague `analyze_content` becomes `extract_web_results` with a web-specific description.
- **Split generic tools into purpose-specific ones** — a do-everything `analyze_document` becomes `extract_data_points`, `summarize_content`, and `verify_claim_against_source`, each with a defined input/output contract.
- **Audit the system prompt for keyword traps** — instructions like "always analyze documents thoroughly" can accidentally bias the model toward any tool with "document" in its name, overriding good descriptions.

Exam tip: When tool misrouting appears with minimal descriptions, the answer is "improve the descriptions." Distractors: few-shot examples (token overhead, doesn't fix the cause), a keyword routing layer (over-engineered), or merging tools (bigger change than a first step warrants).

Task 2.2 — Structured Error Responses for MCP Tools

What it means: When a tool fails, *how* it reports failure determines whether the agent can recover. MCP has an `isError` flag, but a bare "Operation failed" is useless — the agent can't tell whether to retry, rephrase, apologize, or escalate.

The four error categories you must distinguish:

Category	Example	Retryable?	Right agent behavior
Transient	Timeout; service temporarily unavailable	Yes	Retry (subagents should attempt local recovery before bothering the coordinator)
Validation	Malformed order ID; missing required input	No (as-is)	Fix the input and try again
Business	Refund exceeds policy limit	No	Explain the policy to the customer in friendly terms; offer alternatives or escalate
Permission	Agent lacks access rights	No	Escalate or route to an authorized path

Structured error metadata: return `errorCategory`, an `isRetryable` boolean, and a human-readable description. Here is what a well-structured business-rule error looks like:

```
{
  "isError": true,
  "errorCategory": "business_rule",
  "isRetryable": false,
  "message": "Refund of $620.00 exceeds the $500 automatic limit.
             Offer the customer escalation to a supervisor."
}
```

Reading this, the agent knows everything it needs: don't retry (it will fail forever), this is policy rather than a glitch, and here is the friendly path forward to offer the customer. A bare "Operation failed" communicates none of that.

- **Crucial distinction:** "the search failed" (access failure — needs a retry decision) is not the same as "the search succeeded and found nothing" (valid empty result). Conflating them causes wasted retries or, worse, silent data gaps.
- **Local recovery first:** subagents handle transient failures themselves and propagate upward only what they cannot resolve — together with partial results and a record of what was attempted.

Task 2.3 — Distribute Tools Across Agents and Configure `tool_choice`

Fewer tools = better decisions. Giving an agent 18 tools instead of 4–5 measurably degrades selection reliability — every extra tool adds decision complexity. And agents with tools outside their specialty tend to misuse them (a synthesis agent with web-search access starts searching instead of synthesizing).

Scoped access pattern: each subagent gets only its role's tools, plus narrowly scoped cross-role tools for high-frequency needs. The exam's flagship example: the synthesis agent constantly needs small fact checks (85% simple, 15% deep). Right answer: give it a scoped `verify_fact` tool for the simple 85%, and keep routing the complex 15% through the coordinator. Wrong answers: give it *all* search tools (over-provisioning), batch the verifications (creates blocking dependencies), or speculative caching (can't predict needs).

Constrain rather than trust: replace a generic `fetch_url` with a `load_document` tool that validates document URLs — the tool itself enforces the boundary.

tool_choice settings (Claude API):

Setting	Behavior	Use when
"auto"	Model may call a tool or just reply with text	Normal conversation; tool use is optional
"any"	Model MUST call some tool (its choice which)	You need structured output but the right schema/tool depends on the input (e.g., unknown document type)
{"type": "tool", "name": "..."}	Model MUST call that specific tool	A specific step must run first (e.g., force <code>extract_metadata</code> before enrichment); handle later steps in follow-up turns

Task 2.4 — Integrate MCP Servers into Claude Code and Agent Workflows

Config location	Scope	Use for
.mcp.json (in the project repo)	Project-level — shared with the whole team via version control	Team tooling everyone needs (issue tracker, shared databases)
~/.claude.json (home directory)	User-level — personal, not shared	Personal or experimental servers you're trying out

- **Credentials via environment variable expansion:** write `${GITHUB_TOKEN}` in `.mcp.json` instead of the real secret — the config can be safely committed, and each machine supplies its own token.
- **All servers, all at once:** tools from every configured MCP server are discovered at connection time and are simultaneously available — project and personal servers coexist.
- **MCP resources vs. tools:** tools are for *actions*; resources expose *content catalogs* (issue summaries, documentation hierarchies, database schemas) so the agent can see what data exists without burning turns on exploratory tool calls.
- **Rich MCP tool descriptions matter doubly:** if your MCP tool's description is thin, the agent may prefer a familiar built-in (like Grep) over your more capable specialized tool.
- **Buy before build:** use existing community MCP servers for standard integrations (e.g., Jira); write custom servers only for team-specific workflows.

Task 2.5 — Built-In Tools: Read, Write, Edit, Bash, Grep, Glob

Claude Code ships with built-in tools. The exam tests whether you pick the right one for the job:

Tool	What it does	Pick it when...
Grep	Searches file CONTENTS for patterns	Finding all callers of a function, locating an error message or import statement

Tool	What it does	Pick it when...
Glob	Matches file PATHS/names by pattern	Finding files by name or extension, e.g., <code>**/*.test.tsx</code> for every test file
Read	Loads a full file's contents	Following imports, tracing logic after Grep finds the entry point
Write	Writes a complete file	Creating new files; rewriting a file when Edit can't anchor
Edit	Targeted modification via unique text match	Small precise changes — fails if the anchor text isn't unique; fallback is Read + Write
Bash	Runs shell commands	Builds, tests, git operations, scripting

- **Memorize the contrast:** Grep = contents, Glob = file names. This distinction is a near-guaranteed question pattern.
- **Incremental exploration beats bulk reading:** start with Grep to find entry points, then Read to follow the threads — never read every file upfront. For wrapper modules, first identify all exported names, then search for each across the codebase.

7. Domain 3: Claude Code Configuration & Workflows (20%)

The most directly practical domain for daily Claude Code users. It covers the configuration system (where instructions live and who they apply to), reusable commands and skills, plan mode vs. direct execution, iteration techniques, and running Claude Code inside CI/CD pipelines.

Task 3.1 — CLAUDE.md Hierarchy, Scoping, and Modular Organization

CLAUDE.md files are standing instructions Claude Code loads automatically at the start of every session — think of them as the project's operating manual that the new hire reads before touching anything. A real one is just plain text:

```
# Project conventions
- All currency amounts are stored in cents, never dollars
- Every new function needs a matching test
- Use British English in customer-facing text
- Never modify files in the /legacy folder without asking
```

They live at three levels, and the level determines who they apply to (this is where the version-control basics from Section 4.1 pay off):

Level	Location	Applies to	Shared with team?
User	~/.claude/CLAUDE.md	Everything YOU do, in every project	No — personal only
Project	.claude/CLAUDE.md or root CLAUDE.md	Everyone working in this repository	Yes — via version control
Directory	CLAUDE.md inside a subdirectory	Work within that subdirectory	Yes — but bound to that directory

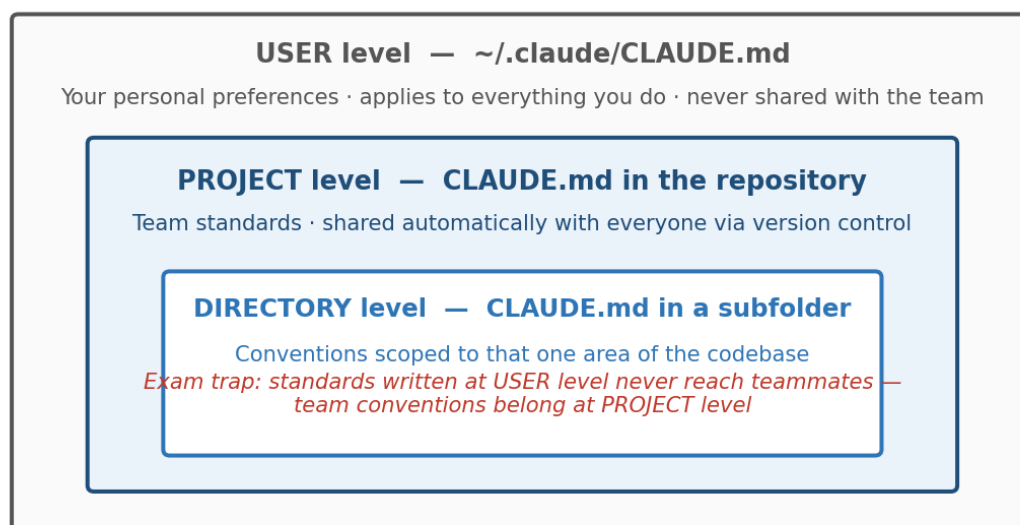


Figure 5 — The three CLAUDE.md scopes nested from broadest (personal) to narrowest (one subfolder).

The classic diagnostic question: a new teammate clones the repo and Claude ignores the team's conventions. Why? The conventions were written in someone's *user-level* file (`~/.claude/CLAUDE.md`), which never travels through version control. Move them to the project level.

- **@import syntax** keeps CLAUDE.md modular — each package's CLAUDE.md imports only the standards files relevant to it.
- **.claude/rules/ directory** is the alternative to one monolithic CLAUDE.md: focused topic files like `testing.md`, `api-conventions.md`, `deployment.md`.
- **/memory command** shows exactly which memory files are currently loaded — the first move when behavior is inconsistent across sessions.

Task 3.2 — Custom Slash Commands and Skills

Slash commands are saved prompts. Put team commands in `.claude/commands/` inside the repo (version-controlled, available to everyone who clones — the correct answer for "the whole team should get /review"). Personal commands go in `~/.claude/commands/`.

Skills live in `.claude/skills/` as folders with a SKILL.md playbook, configured via frontmatter (settings at the top of the file):

Frontmatter option	What it does	Example use
context: fork	Runs the skill in an isolated sub-agent context so its verbose output doesn't pollute the main conversation	A codebase-analysis skill that produces pages of findings; a brainstorming skill generating throwaway alternatives
allowed-tools	Restricts which tools the skill may use while running	Limit a formatting skill to file writes only, preventing destructive actions
argument-hint	Prompts the developer for required parameters when the skill is invoked without arguments	A /deploy skill that needs an environment name

Skills vs. CLAUDE.md — the decision rule: CLAUDE.md is for **always-loaded universal standards**; skills are for **on-demand, task-specific workflows**. Want a personal variant of a team skill? Create it in `~/.claude/skills/` under a *different name* so teammates are unaffected.

Task 3.3 — Path-Specific Rules (Conditional Convention Loading)

What it means: Rule files in `.claude/rules/` can carry YAML frontmatter (the settings block from Section 4.1) with a `paths` field containing glob patterns (also Section 4.1). The rule loads *only* when Claude is editing files that match — saving tokens and avoiding irrelevant context. A complete, working rule file:

```
---
paths: ["**/*.test.tsx"]
---
When writing or editing tests:
- Test names describe the behavior being verified
- Mock external services; never call real APIs
- Cover the empty-input and error cases first
```

Save that as `.claude/rules/testing.md` and the testing conventions now apply automatically to every test file in the project — and *only* to test files.

Why this beats directory CLAUDE.md files for scattered conventions: test files sit next to the code they test, all over the codebase (Button.test.tsx beside Button.tsx). A directory-bound CLAUDE.md can't cover them all; a single rule with `paths: ["**/*.test.tsx"]` applies the testing conventions to every test file regardless of location — automatically and deterministically.

Analogy: Path rules are motion-sensor lights: they switch on exactly where you're working. A monolithic CLAUDE.md is leaving every light in the building on.

Exam tip: "Conventions for files spread across many directories, applied automatically" → path-scoped rules with glob patterns. Distractors: one big CLAUDE.md (relies on inference), skills (manual/optional invocation), per-directory CLAUDE.md files (directory-bound).

Task 3.4 — Plan Mode vs. Direct Execution

Mode	Designed for	Examples
Plan mode	Complex tasks: large-scale changes, multiple valid approaches, architectural decisions, multi-file modifications — explore and design BEFORE changing anything	Monolith → microservices restructuring; a library migration touching 45+ files; choosing between integration approaches with different infrastructure needs
Direct execution	Simple, well-scoped changes you already understand	Single-file bug fix with a clear stack trace; adding one date-validation conditional

- **Plan mode prevents costly rework** by surfacing dependencies before you commit to changes. If the requirements already state the complexity (dozens of files, service-boundary decisions), "start direct and switch to plan mode if it gets hard" is a trap answer — the complexity isn't hypothetical.
- **The Explore subagent** isolates verbose discovery output and returns summaries, preserving the main conversation's context during multi-phase tasks.
- **Combine the modes:** plan mode for investigation, then direct execution to implement the agreed plan.

Task 3.5 — Iterative Refinement Techniques

Four techniques for progressively improving Claude's output:

- **Concrete input/output examples (2–3):** the most effective way to communicate a transformation when prose descriptions are interpreted inconsistently. Show the exact before → after.
- **Test-driven iteration:** write the test suite first (expected behavior, edge cases, performance), then iterate by sharing test failures — the failures become precise, objective feedback.
- **The interview pattern:** have Claude interview YOU before implementing in an unfamiliar domain — it surfaces considerations you hadn't anticipated (cache invalidation strategy? failure modes?).
- **Batch interacting fixes; sequence independent ones:** if multiple issues interact, describe them all in one detailed message so the fix accounts for the interactions; independent issues can be fixed one at a time.

Task 3.6 — Claude Code in CI/CD Pipelines

Running Claude Code with no human at the keyboard requires specific flags and context discipline:

Mechanism	Purpose
-p (or --print) flag	Non-interactive mode: process the prompt, print the result, exit. Without it the CI job hangs forever waiting for input — the exam's sample question. (CLAUDE_HEADLESS and --batch do not exist.)
--output-format json + --json-schema	Forces machine-parseable structured findings — so the pipeline can automatically post them as inline PR comments
CLAUDE.md in CI	Supplies project context to the automated run: testing standards, fixture conventions, review criteria — directly improves test-generation quality and reduces low-value output
Independent review instance	The session that GENERATED code is biased when reviewing it (it carries its own reasoning); a fresh instance without that context catches more issues
Prior findings in context on re-review	When re-running review after new commits, include earlier findings and instruct Claude to report only new or still-unaddressed issues — prevents duplicate comments
Existing tests in context	Provide current test files so test generation doesn't suggest scenarios already covered

8. Domain 4: Prompt Engineering & Structured Output (20%)

How to get precise, consistent, machine-readable output: explicit criteria, worked examples, schema-enforced structure, validation loops, batch economics, and review architectures.

Task 4.1 — Explicit Criteria to Reduce False Positives

The principle: vague instructions produce vague judgment. "Check that comments are accurate" invites noise; "flag comments only when the claimed behavior contradicts the actual code behavior" is testable and precise.

- **"Be conservative" doesn't work.** General modifiers like "only report high-confidence findings" fail to improve precision. Specific categorical criteria do: report bugs and security issues; skip minor style and local patterns.
- **False positives are trust killers:** a category with high false positives undermines developers' confidence in the *accurate* categories too. Tactical move: temporarily disable the noisy category while you improve its prompt.
- **Severity needs anchors:** define each severity level with concrete code examples to get consistent classification.

Analogy: A security guard told "report anything suspicious" floods you with junk reports. Told "report propped-open doors after 8pm and unbadged individuals in the server room," the reports become rare and worth acting on.

Task 4.2 — Few-Shot Prompting

What it means: include 2–4 worked examples in the prompt showing input → correct output. This is the most effective technique when detailed instructions alone still produce inconsistent results, because the model generalizes the demonstrated *judgment* to novel cases — it doesn't just match the literal examples.

What few-shot actually looks like (two examples placed inside an invoice-extraction prompt):

```
Example 1
Input: "Pd $1,200.50 to Acme Corp on 3/4/25"
Output: {"vendor": "Acme Corp", "amount": 1200.50,
        "date": "2025-03-04"}

Example 2
Input: "Invoice received from Beta LLC, amount to be confirmed"
Output: {"vendor": "Beta LLC", "amount": null, "date": null}
```

Example 1 silently teaches three normalization rules (strip the \$ and comma, expand the abbreviated date, use the standard date format). Example 2 teaches the most important behavior of all: **when information is genuinely missing, write null — do not invent a number.** No paragraph of prose communicates that boundary as reliably as one demonstrated case.

Where few-shot shines (per the exam guide):

- **Ambiguous-case handling:** show the reasoning for why one action was chosen over the plausible alternative (tool selection for vague requests, branch-level test coverage gaps).

- **Format consistency:** demonstrate the exact output shape — location, issue, severity, suggested fix.
- **False-positive reduction:** pair examples of acceptable patterns with genuine issues so the model learns the boundary.
- **Hallucination reduction in extraction:** show correct extraction from varied document structures — inline citations vs. bibliographies, informal measurements, narrative vs. tabular formats — including correct null handling when a field is absent.

Task 4.3 — Structured Output via Tool Use and JSON Schemas

The reliable trick: to get guaranteed schema-compliant JSON, define a "tool" whose input parameters ARE your desired output schema, and read the data from the model's `tool_use` call. This eliminates JSON *syntax* errors entirely.

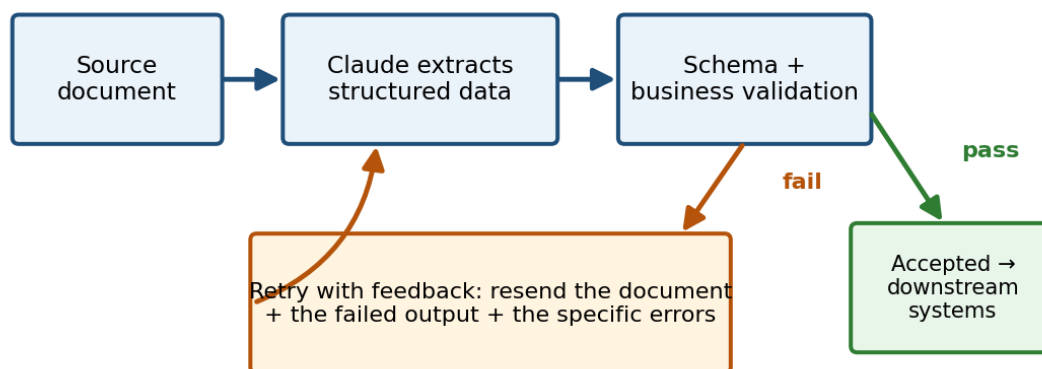
The limit you must know: schemas eliminate syntax errors but NOT **semantic** errors — line items that don't sum to the stated total, or values landing in the wrong field, still happen and need validation (Task 4.4).

Schema design rules that prevent hallucination:

- **Make fields nullable/optional** when source documents may genuinely lack the information. If a field is required, the model will *fabricate a value* to satisfy the schema — the single most important schema-design fact on the exam.
- **Extensible enums:** add an `"unclear"` value for ambiguous cases and an `"other"` value paired with a free-text detail field, so real-world variety doesn't get force-fitted into wrong categories.
- **Pair normalization rules with strict schemas:** tell the model how to standardize inconsistent source formats (dates, currencies) alongside the schema.

Combine with the `tool_choice` table from Task 2.3: `"any"` guarantees structured output when the document type (and hence the right schema) is unknown; forced selection guarantees a *specific* extraction runs.

Task 4.4 — Validation, Retry, and Feedback Loops



Retries fix format and structure — they can never conjure information absent from the source

Figure 6 — The retry-with-feedback loop: failures travel back WITH the specific errors attached.

- **Retry-with-error-feedback:** on validation failure, send a follow-up containing the original document + the failed extraction + the specific validation errors. Specific feedback steers self-correction; blind retries don't.
- **Know when retry is futile:** retries fix format and structural errors; they can NEVER fix information that is simply absent from the source (e.g., it lives in an external document you didn't provide). Recognizing futile-retry situations is a tested skill.
- **Self-correcting schema design:** extract `calculated_total` alongside `stated_total` so discrepancies flag themselves; add a `conflict_detected` boolean for internally inconsistent sources.
- **Feedback loops for review systems:** add a `detected_pattern` field to each finding so when developers dismiss findings, you can analyze which code constructs drive false positives — systematically, not anecdotally.

Task 4.5 — Batch Processing Strategies

The Message Batches API in one line: 50% cheaper, results within up to 24 hours, **no guaranteed latency SLA**, and no multi-turn tool calling within a request.

Workload	Right API	Why
Pre-merge code check (developers waiting)	Synchronous (real-time)	Blocking workflow — "usually fast" isn't good enough when there's no SLA
Overnight technical-debt report	Batch	Latency-tolerant; pocket the 50% savings
Weekly compliance audit / nightly test generation	Batch	Same — nobody is waiting on the result

- **custom_id** correlates each request with its response — and lets you resubmit *only the failed documents* (with fixes such as chunking oversized ones) instead of rerunning the whole batch.
- **SLA math:** with a 24-hour batch window, guaranteeing a 30-hour turnaround means submitting batches at least every ~4–6 hours (the guide's example: 4-hour submission windows guarantee a 30-hour SLA).
- **Refine before you scale:** tune the prompt on a small sample first; first-pass success across 100,000 documents is far cheaper than iterative resubmission.

Task 4.6 — Multi-Instance and Multi-Pass Review Architectures

- **Self-review is structurally weak:** a model reviewing code in the same session that generated it carries its own reasoning context and is unlikely to question its own decisions. A **fresh, independent instance** (no prior reasoning) catches subtler issues than self-review instructions or extended thinking ever will.
- **Multi-pass beats single-pass for large reviews:** per-file passes for local issues + a separate cross-file integration pass — avoids attention dilution and contradictory findings (the 14-file PR sample question; bigger context windows do NOT fix attention quality, and consensus voting across runs suppresses real intermittent catches).
- **Confidence-annotated verification passes:** have the model self-report a confidence score next to each finding to enable calibrated routing of human review attention.

Analogy: Authors don't copy-edit their own manuscripts — they read what they meant to write. Publishers hire fresh eyes. Same model, different session, no shared reasoning = fresh eyes.

9. Domain 5: Context Management & Reliability (15%)

The smallest domain by weight but conceptually woven through every scenario: how to keep critical facts alive in long interactions, escalate wisely, propagate errors usefully, survive huge codebases, calibrate human review, and preserve "who said what" through synthesis.

Task 5.1 — Preserve Critical Information Across Long Interactions

Three failure physics you must internalize:

- **Progressive summarization erodes precision:** each round of summarizing strips detail. Watch it happen:

```
Original:    "Customer Dana Lee (order #88231) was promised a
              $847.50 refund by March 3 after a second damaged
              delivery."
Summary 1:   "Dana Lee was promised a refund for delivery issues."
Summary 2:   "A customer had delivery problems; a refund was
              discussed."
```

Two rounds in, every actionable fact — name, order number, amount, deadline — is gone, yet each individual summary looked reasonable. Numbers, percentages, dates, and customer-stated expectations are always the first casualties.

- **Lost in the middle:** models reliably process the beginning and end of long inputs but may skip the middle. Mitigation: put a key-findings summary at the TOP and organize details under explicit section headers.
- **Tool-output bloat:** an order lookup returns 40+ fields when 5 matter; multiply by dozens of calls and the context drowns. Trim tool outputs to relevant fields *before* they accumulate.

The signature pattern — the "case facts" block: extract transactional facts (amounts, dates, order numbers, statuses) into a persistent structured block included in every prompt, *outside* the summarized history. Summaries can blur; the facts block cannot.

- **For multi-agent handoffs:** require subagents to return structured data with metadata (dates, source locations, methodological context) — and when downstream agents have tight context budgets, have upstream agents return key facts + citations + relevance scores instead of verbose prose and reasoning chains.
- **Also remember:** the API is stateless — you must pass the complete conversation history in each request to maintain coherence.

Task 5.2 — Escalation and Ambiguity Resolution

Legitimate escalation triggers: (1) the customer explicitly asks for a human, (2) policy is ambiguous or silent on the request (e.g., competitor price-matching when policy only covers own-site adjustments), (3) the agent cannot make meaningful progress.

Signal	Reliable trigger?	Why
Customer explicitly demands a human	Yes — honor immediately	Do not "investigate first"; that disrespects an explicit preference
Customer is frustrated but the issue is simple	No — resolve it	Acknowledge the frustration, offer the fix; escalate only if they reiterate

Signal	Reliable trigger?	Why
Negative sentiment score	No	Sentiment does not correlate with case complexity
Model self-reported confidence score	No	LLM confidence is poorly calibrated — the agent is precisely most overconfident on the hard cases
Policy gap / exception required	Yes	The agent must not invent policy

- **The calibration fix:** when an agent escalates easy cases while fumbling hard ones (55% vs. an 80% resolution target), add **explicit escalation criteria with few-shot examples** to the system prompt — not confidence thresholds, not a classifier model, not sentiment analysis.
- **Multiple customer matches:** ask for additional identifiers. Never pick "the most likely" account by heuristic — that is how wrong-account refunds happen.

Task 5.3 — Error Propagation Across Multi-Agent Systems

When a subagent fails (say, a web-search timeout), what flows back to the coordinator determines whether intelligent recovery is possible:

Approach	Verdict	Why
Structured error context: failure type, attempted query, partial results, alternative approaches	CORRECT	The coordinator can decide: retry modified, try an alternative source, or proceed with partials and annotate the gap
Retry internally, then return generic "search unavailable"	Anti-pattern	Local retries are good; the GENERIC status hides everything the coordinator needs
Return empty results marked as success	Anti-pattern	Silent suppression — the report ships with invisible holes
Propagate the exception; terminate the whole workflow	Anti-pattern	One recoverable failure kills the entire research run

- **Coverage annotations:** synthesis output should mark which findings are well-supported and which topic areas have gaps due to unavailable sources — honest uncertainty beats false completeness.

Task 5.4 — Context Management in Large Codebase Exploration

- **Recognize context degradation:** in long sessions the model starts answering from "typical patterns" instead of the specific classes it actually discovered earlier — a sign its working memory has been squeezed.
- **Scratchpad files:** have the agent write key findings to a file and consult it for later questions — external memory that survives context pressure.
- **Delegate the verbose work:** spawn subagents for noisy investigations ("find all test files," "trace refund-flow dependencies") so only summaries return to the main agent, which preserves clean high-level coordination.
- **Phase summaries:** summarize each exploration phase before spawning the next phase's subagents, injecting the summary into their initial context.

- **Crash recovery via manifests:** each agent exports structured state to a known location; on resume, the coordinator loads the manifest and injects it into agent prompts — no restart from zero.
- `/compact` condenses the conversation when extended exploration fills the context with verbose discovery output.

Task 5.5 — Human Review Workflows and Confidence Calibration

- **Aggregate metrics hide pockets of failure:** "97% overall accuracy" can mask 60% accuracy on one document type or field. Validate accuracy **by document type AND field segment** before automating anything.
- **Stratified random sampling:** keep auditing samples from HIGH-confidence extractions too — it measures the true error rate where you've stopped looking and detects novel error patterns early.
- **Calibrated confidence routing:** have the model output field-level confidence scores, calibrate thresholds against a labeled validation set, then route low-confidence or ambiguous/contradictory-source extractions to humans — spending scarce reviewer capacity where it matters most.

Analogy: Airport security: most travelers flow through standard screening (high-confidence), a calibrated system selects some for additional checks (low-confidence routing), and random audits of the fast lane (stratified sampling) verify the system itself still works.

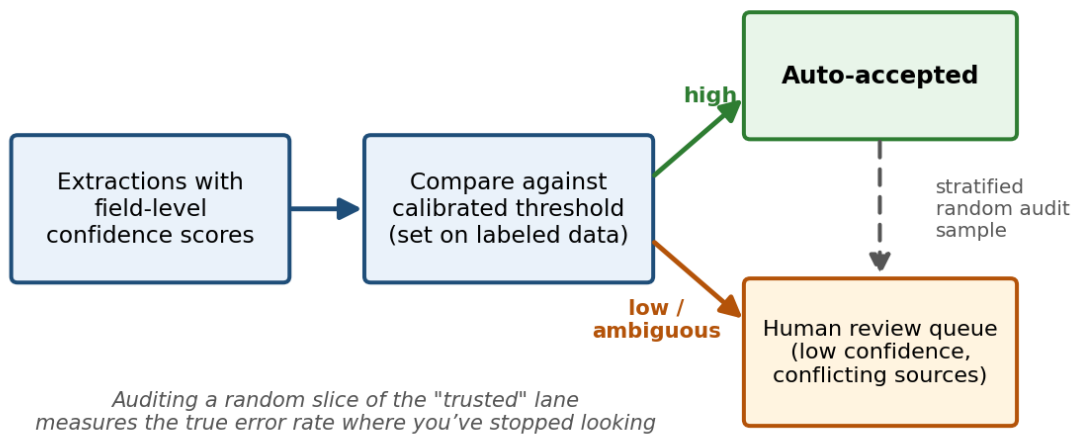


Figure 7 — Calibrated confidence routing, with the audit loop that keeps the "trusted" lane honest.

Task 5.6 — Information Provenance and Uncertainty in Multi-Source Synthesis

- **Attribution dies in summarization** unless you force structure: subagents must output **claim-source mappings** (claim + source URL/document name + relevant excerpt) that downstream agents are required to preserve and merge.
- **Conflicting statistics from credible sources:** annotate the conflict with both values and their sources — never arbitrarily pick one. Let the coordinator decide reconciliation before synthesis.

- **Temporal traps:** a 2021 figure and a 2024 figure are not a "contradiction" — they're a trend. Require publication/collection dates in structured outputs so time differences aren't misread as disagreement.
- **Report structure:** explicitly separate well-established findings from contested ones, preserving the original sources' characterizations and methodological context.
- **Render content natively:** financial data as tables, news as prose, technical findings as structured lists — don't flatten everything into one uniform format.

10. Sample Question Walkthroughs: Learning the Exam's Logic

The exam guide publishes twelve sample questions. Rather than repeat them, here is the **reasoning pattern** behind each — the transferable logic that will let you answer questions you've never seen.

Sample question (paraphrased)	Correct logic	Why the distractors fail
Q1: Agent skips identity check before refunds in 12% of cases	Programmatic prerequisite gate blocking refund/order tools until get_customer verifies the ID	Prompt wording and few-shot examples are probabilistic; a routing classifier solves availability, not ordering
Q2: Agent confuses two tools with minimal descriptions	Expand the tool descriptions (inputs, examples, edge cases, boundaries) — the low-effort, root-cause first step	Few-shot adds tokens without fixing the cause; a keyword router is over-engineered; merging tools is too big for a "first step"
Q3: Agent escalates easy cases, attempts hard ones (55% vs 80% target)	Explicit escalation criteria + few-shot examples in the system prompt	Self-reported confidence is miscalibrated; a classifier model is over-engineered; sentiment ≠ complexity
Q4: Team-wide /review command	.claude/commands/ in the repo — version control distributes it	~/claude/commands/ is personal; CLAUDE.md isn't for commands; .claude/config.json commands array doesn't exist
Q5: Monolith → microservices restructuring	Plan mode: explore, understand dependencies, design before changing	Complexity is stated upfront — "switch later if it gets hard" and "detailed upfront instructions without exploring" both ignore that
Q6: Different conventions per code area; test files scattered everywhere	.claude/rules/ with glob-pattern frontmatter (e.g., <code>**/*.test.tsx</code>)	Monolithic CLAUDE.md relies on inference; skills need invocation; per-directory CLAUDE.md can't follow scattered files
Q7: Research report covers only visual arts; coordinator logs show 3 narrow subtasks	Root cause = coordinator's narrow task decomposition	Subagents executed their assignments correctly — blaming them ignores the logs
Q8: Web-search subagent times out; design the failure flow	Structured error context: failure type, attempted query, partial results, alternatives	Generic status hides context; empty-as-success suppresses; full termination overreacts
Q9: Synthesis agent needs frequent small fact checks (85% simple)	Scoped verify_fact tool for the common case; coordinator path for the complex 15%	Batching blocks dependent steps; all search tools over-provisions; speculative caching can't predict needs
Q10: CI job hangs waiting for input	claude -p "..." — the non-interactive print flag	CLAUDE_HEADLESS and --batch don't exist; stdin

Sample question (paraphrased)	Correct logic	Why the distractors fail
		redirection doesn't address Claude Code's syntax
Q11: Cut API costs: pre-merge check + overnight report	Batch the overnight report only; keep real-time for the blocking check	Batch has no latency SLA — unacceptable for blocking; custom_id solves ordering, so that fear is a misconception
Q12: 14-file PR review is inconsistent and contradictory	Split into per-file passes + a cross-file integration pass	Bigger context ≠ better attention; forcing small PRs shifts the burden; 2-of-3 consensus suppresses real intermittent catches

Pattern recognition: Notice the rhythm: root cause + simplest reliable mechanism wins. Distractors are "more prompting where enforcement is needed," "more infrastructure where prompting suffices," and "features that don't exist." When you spot an invented flag or config file, eliminate it instantly.

11. Putting It to Work: Claude Code Habits for Non-Developers

Everything above doubles as a practical playbook. Here are the habits that improve your day-to-day Claude Code results immediately — each mapped to the exam domain it reinforces.

Habit	What to do	Domain
Write a project CLAUDE.md	Put your standards, conventions, and context in the repo's CLAUDE.md so every session (and every teammate) starts informed	D3
Keep it modular	Split big instruction files into .claude/rules/ topic files; add paths: globs so rules load only where relevant	D3
Save repeated prompts as slash commands	Anything you've typed three times becomes a command in .claude/commands/	D3
Plan before big changes	Use plan mode for anything multi-file or architectural; direct execution for small known fixes	D3
Show, don't describe	Give 2–3 input → output examples instead of longer prose when results are inconsistent	D4
Let Claude interview you	Before a build in an unfamiliar area, ask Claude to ask YOU clarifying questions first	D3
Demand structure for data	Ask for output in a defined schema with explicit "unknown/other" options so gaps aren't papered over with invented values	D4
Trim and persist	In long sessions, ask Claude to maintain a running "key facts" note (or scratchpad file) and use /compact when the chat gets bloated	D5
Fresh eyes for review	Review important generated work in a NEW session — never the one that produced it	D4
Escalate by rule, not vibe	Define explicit criteria for when automation hands off to a human; write down examples	D5

12. One-Page Cheat Sheet: Facts to Memorize

Topic	The fact
stop_reason	"tool_use" = run tool & continue loop; "end_turn" = done. Never parse text or rely on iteration caps to terminate
Subagent memory	Isolated — context must be passed explicitly in the prompt; coordinator needs "Task" in allowedTools
Parallel subagents	Multiple Task calls in ONE coordinator response
Enforcement	Guaranteed rules → hooks/gates (deterministic); preferences → prompts (probabilistic)
Hooks	PostToolUse = normalize results after; interception = block policy violations before (e.g., refunds > \$500)
Tool selection	Driven by descriptions; ~4–5 well-described tools beat 18; first fix = better descriptions
tool_choice	"auto" = optional; "any" = must call some tool; {"type":"tool","name":...} = must call that tool
Errors	Categories: transient / validation / business / permission; return errorCategory + isRetryable; empty result ≠ failure
MCP scoping	.mcp.json = project/shared; ~/.claude.json = personal; secrets via \${ENV_VAR}; resources = content catalogs, tools = actions
Built-ins	Grep = file CONTENTS; Glob = file NAMES/paths; Edit needs unique anchor text (fallback Read + Write)
CLAUDE.md levels	User (~/.claude/, personal) · Project (repo, shared) · Directory (scoped); @import for modularity; /memory to debug
Commands & skills	.claude/commands/ = shared; ~/.claude/commands/ = personal; skill frontmatter: context: fork, allowed-tools, argument-hint
Path rules	.claude/rules/ + YAML paths: ["glob"] → loads only when editing matching files
Plan mode	Architectural / multi-file / multiple approaches → plan; single well-scoped fix → direct
CI/CD	-p / --print = non-interactive; --output-format json + --json-schema = structured CI output
Schemas	Tool use kills JSON syntax errors, NOT semantic errors; nullable fields prevent fabrication; enums need "unclear"/"other"+detail
Retry	Works for format/structure errors; futile when the information is absent from the source
Batches API	50% cheaper, ≤24h window, NO latency SLA, no multi-turn tool calling; custom_id correlates & enables partial resubmission
Review	Independent fresh instance > self-review; large PRs → per-file passes + integration pass

Topic	The fact
Context	"Case facts" block survives summarization; lost-in-the-middle → summary first + section headers; trim tool outputs early
Escalation	Triggers: explicit human request (honor immediately), policy gap, no progress. NOT sentiment, NOT self-confidence
Provenance	Claim-source mappings preserved through synthesis; annotate conflicts with both sources; include dates to avoid temporal "contradictions"

13. A Two-Week Study Plan

Days	Focus	Activity
1–2	Foundations	Read Chapters 1–4 of this tutorial (non-technical members: spend extra time on Section 4.1); skim the official guide's scenario descriptions; make flashcards from the glossary
3–5	Domains 1 & 2 (45% of exam)	Study Chapters 5–6; in Claude Code, watch the loop in action — ask it to use tools and observe the think→act→observe rhythm
6–7	Domain 3 hands-on	Create a real project CLAUDE.md, one slash command, and one path-scoped rule; try plan mode vs. direct execution on tasks of different sizes
8–9	Domains 4 & 5	Study Chapters 8–9; practice writing few-shot examples and explicit criteria; design a schema with nullable fields and an "other" enum
10–11	Sample questions	Work Chapter 10 walkthroughs; for each, articulate WHY each distractor fails before reading the explanation
12–13	Practice exam	Take the official practice exam; re-study every domain where you missed questions
14	Final review	Re-read the cheat sheet (Chapter 12) and the three recurring themes (Chapter 1); rest

Good luck — and remember the exam's underlying question is always the same: what is the simplest mechanism that reliably fixes the actual root cause?